

хекслет
колледж

ПМ 1

Разработка программного обеспечения

ТЕМА ЗАНЯТИЯ

Жизненный цикл ПО



Изучаем



Разбираем



Применяем

Цель занятия

Что должно измениться в понимании студента после изучения темы



Хекслет
колледж



Сформировать целостное представление о **жизненном цикле программного обеспечения**, его этапах, процессах и значении для разработки, внедрения, эксплуатации и сопровождения программных продуктов.



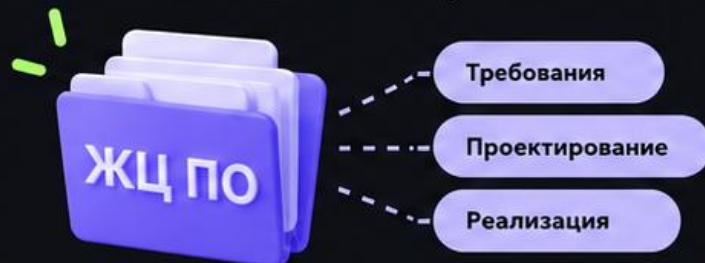
01 Понимать

- что такое ЖЦ ПО
- почему ЖЦ ПО охватывает путь продукта от идеи до снятия с эксплуатации
- зачем разработчику знать этапы жизненного цикла



02 Различать

- процессы и стадии ЖЦ ПО
- основные стадии: требования, проектирование, реализация, тестирование
- эксплуатацию, сопровождение и вывод из эксплуатации



03 Объяснять

- роль требований в успехе проекта
- значение проектирования, тестирования и сопровождения
- связь между качеством ПО и соблюдением этапов ЖЦ



Хекслет
колледж

Жизненный цикл программного обеспечения

Что такое ЖЦ ПО и почему этот термин важен для разработчика



хекслет
колледж



Ключевая идея

Жизненный цикл ПО (ЖЦ ПО) — это весь путь программного продукта: **от идеи до вывода из эксплуатации.**

Он включает определение требований, разработку, тестирование, внедрение, эксплуатацию, сопровождение и завершение использования.



01 Определение

- ЖЦ ПО описывает этапы и процессы, через которые проходит программный продукт.
- Он помогает планировать разработку, тестирование, внедрение и поддержку.
- ЖЦ нужен, чтобы продукт был качественным, управляемым и полезным пользователю.



02 Стандарт

- Ключевой международный стандарт: **ISO/IEC/IEEE 12207.**
- Стандарт описывает процессы жизненного цикла программного обеспечения.
- Он не навязывает одну модель разработки, а задаёт общую структуру процессов.



Исторически первая редакция стандарта была опубликована в 1995 году, позже стандарт обновлялся.

03 Что включает ПО

- программы и модули
- данные
- процедуры и правила работы
- документацию



ПО — это **не только код**, но и всё, что нужно для его использования и сопровождения.



хекслет
колледж

Жизненный цикл ISO/IEC/IEEE 12207

Как стандарт описывает группы процессов жизненного цикла ПО



Хекслет
колледж



Ключевая идея

Стандарт ISO/IEC/IEEE 12207 описывает процессы жизненного цикла программного обеспечения. Для изучения удобно рассматривать их как **3 укрупнённые группы**: основные, вспомогательные и организационные.



01

Основные процессы

- приобретение
- поставка
- разработка
- эксплуатация
- сопровождение



Связаны с созданием, поставкой и использованием ПО.

02

Вспомогательные процессы

- документирование
- управление конфигурацией
- обеспечение качества
- верификация и валидация
- совместная оценка и аудит
- решение проблем



Поддерживают качество, контроль и согласованность продукта.

03

Организационные процессы

- управление
- создание инфраструктуры
- совершенствование процессов
- обучение персонала



Создают условия для эффективной разработки и сопровождения.



Важно: в разных редакциях стандарта состав и группировка процессов уточнялись, поэтому в учебных материалах часто используют укрупнённое представление.

Взаимосвязь между процессами ЖЦ ПО



хекслет
колледж

Процессы ЖЦ ПО, регламентируемые стандартом ISO/IEC 12207, могут использоваться различными организациями в конкретных проектах самым различным образом.



Процессы ЖЦ ПО **взаимосвязаны** и могут использоваться в проектах **в различном сочетании** в зависимости от целей, масштаба и контекста.



Модели и стадии ЖЦ ПО

Модель жизненного цикла определяет, как выполняются процессы, действия и задачи на протяжении всего жизненного цикла программного обеспечения.

Что такое модель ЖЦ ПО?

Модель ЖЦ ПО — это структура, определяющая последовательность выполнения и взаимосвязи процессов, действий и задач на протяжении жизненного цикла ПО.



От чего зависит выбор модели?



Специфика проекта

Тип продукта, предметная область, требования заказчика



Масштаб

Размер команды, бюджет, сроки разработки



Сложность проекта

Количество функций, интеграции, технические риски



Специфика условий

Регулирующие требования, среда разработки, распределенные команды



ISO/IEC 12207 не предлагает

конкретную модель ЖЦ и методы разработки ПО. Стандарт описывает структуру процессов, но не конкретизирует, как выполнять действия и задачи.

Примеры моделей ЖЦ ПО

Каскадная модель



Последовательное выполнение стадий

Итеративная модель



Повторение циклов с постепенным улучшением

Инкрементная модель



Постепенное наращивание функциональности

Гибкие методологии (Agile, Scrum)



Короткие итерации, быстрая обратная связь



Модели ЖЦ помогают **адаптировать** процессы разработки **под конкретные цели, ресурсы и условия проекта.**



Стадии ЖЦ ПО

Процесс создания ПО состоит из последовательных стадий (фаз), каждая из которых завершается конкретным результатом.



Стадии выделяются для рационального планирования и организации работ, чтобы достичь заданных результатов.

хекслет
колледж



Процесс создания ПО

Совокупность упорядоченных во времени, взаимосвязанных и объединенных в стадии (фазы) работ, выполнение которых необходимо и достаточно для создания ПО, соответствующего заданным требованиям.



Стадия (фаза) создания ПО

Часть процесса создания ПО, ограниченная некоторыми временными рамками и заканчивающаяся выпуском конкретного продукта (моделей ПО, программных компонентов, документации и др.), определяемого заданными для данной стадии требованиями.



Зачем выделяют стадии?

Стадии создаются по соображениям рационального планирования и организации работ, чтобы каждая стадия заканчивалась измеримым и полезным результатом.

Пример: каждая стадия дает свой результат



Четкое разделение на стадии помогает **управлять проектом, контролировать качество и достигать поставленных целей.**



Стадия формирования требований

1/4

Одна из ключевых стадий жизненного цикла ПО

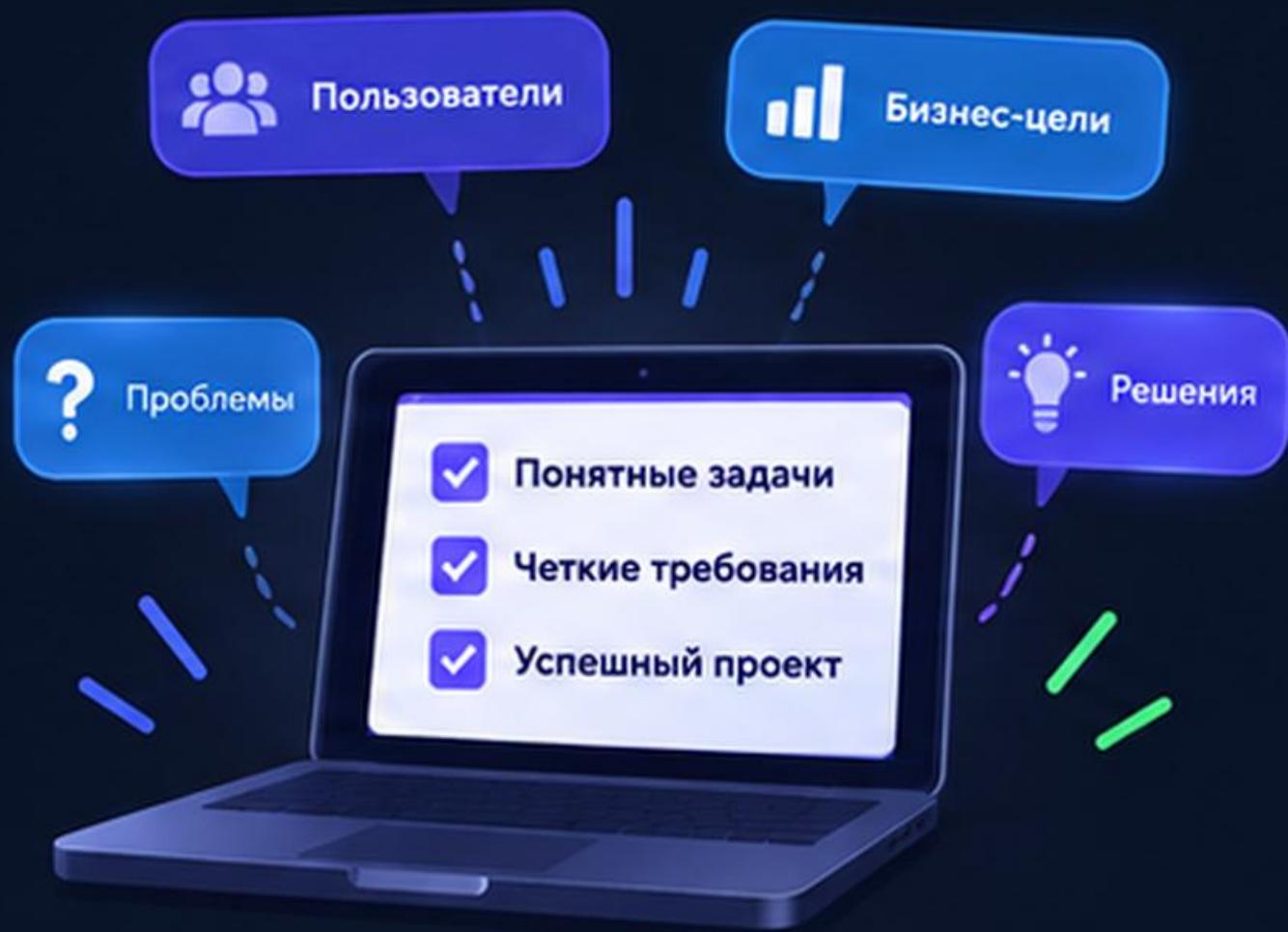


Что это?

Стадия формирования требований — это этап, на котором определяются потребности пользователей и заказчика, описывается, какое ПО должно быть создано и какие задачи оно должно решать.



От качества требований зависит успех всего проекта.



Почему эта стадия так важна?

2/4

— Требования определяют направление всей дальнейшей работы



Задаёт цели и границы проекта

Помогает понять,
что именно нужно
разработать,
а что не входит
в область проекта.



Объединяет всех участников

Единое понимание
между заказчиком,
пользователями
и командой
разработки.



Снижает риски

Позволяет выявить
проблемы и
ограничения на
раннем этапе,
когда их проще
и дешевле решить.



Влияет на качество продукта

Чем точнее
и полнее требования,
тем выше вероятность,
что итоговое ПО будет
удовлетворять
потребности
пользователей.



Качественные требования снижают риск переделок и помогают управлять сроками, стоимостью и качеством продукта.

Этапы формирования требований

3/4

Один из вариантов организации работ по формированию требований

1

Планирование работ



- Определение целей разработки
- Предварительная экономическая оценка
- Построение плана-графика выполнения работ
- Создание и обучение совместной рабочей группы

2

Обследование деятельности организации



- Выявление требований к будущей системе
- Определение структуры организации
- Анализ функций по подразделениям и сотрудникам
- Выявление информационных потоков и взаимодействий
- Анализ существующих средств автоматизации

3

Построение модели деятельности



Обработка материалов обследования и построение двух видов моделей:

AS-IS

«Как есть»

Текущее состояние организации

TO-BE

«Как должно быть»

Желаемое состояние с учетом новых технологий

i

AS-IS и TO-BE — полезные техники бизнес-анализа, но ISO/IEC/IEEE 12207 не требует именно такой последовательности работ.

Результаты стадии формирования требований 4/4

На выходе — согласованный набор требований и связанных спецификаций

Спецификации

- ☒ Функциональные требования
- ☒ Нефункциональные требования
- ☒ Интерфейсные требования



Функциональные требования

Что должно делать ПО: функции, правила и сценарии использования.



Нефункциональные требования

Производительность, надёжность, безопасность, совместимость, доступность и другие характеристики качества.



Интерфейсные требования

Взаимодействие пользователя, внешних систем и компонентов: интерфейсы, форматы данных, протоколы.



Требования должны быть согласованными, однозначными, осуществимыми, проверяемыми и трассируемыми.

Архитектурное проектирование

От требований — к структуре программной системы



Требования



Архитектура



Компоненты
и интерфейсы



Архитектурное
описание

Что определяем на уровне архитектуры

01

Компоненты и ответственность

какие крупные части есть у системы и за что отвечает каждая

02

Интерфейсы и взаимодействия

как компоненты, пользователи и внешние системы обмениваются данными

03

Потоки данных и интеграции

откуда приходят данные, куда идут и где преобразуются

04

Ограничения и качественные атрибуты

производительность, безопасность, надёжность, масштабируемость и др.

Главный вопрос

Как будет устроена система, чтобы выполнить требования?

UI



API



DB

Сервис



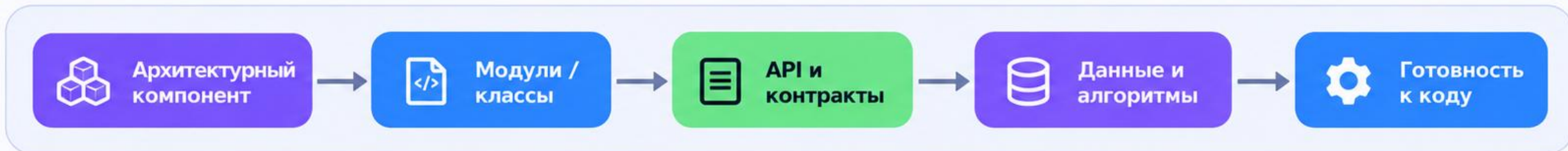
Результат: архитектурное описание: схемы, компоненты, интерфейсы, потоки данных, ключевые решения и ограничения.



Требования отвечают «ЧТО нужно?», а архитектурное проектирование — «КАК организовать решение на структурном уровне?»

Детальное проектирование ПО

От архитектурных компонентов — к реализации



Что уточняем в детальном проектировании



Модули и ответственность

структура внутри компонентов, зависимости и зоны ответственности



API / интерфейсы

операции, входные и выходные данные, контракты и обработка ошибок



Модели данных

структуры, схемы, связи, форматы хранения и обмена



Алгоритмы и логика

ключевые сценарии, правила обработки, состояния и исключения



Размещение и ресурсы

только там, где это важно: среда выполнения, память, производительность, инфраструктура

На выходе



Детальное описание,

достаточное для реализации и согласованное с архитектурой и требованиями.

Проверяем перед кодированием

- ✓ согласованность решений
- ✓ трассировка к требованиям
- ✓ реализуемость и тестируемость
- ✓ понятность интерфейсов и данных

Design Review → Реализация

Проектирование может уточняться итеративно; состав артефактов зависит от масштаба и контекста проекта.

Стадия реализации ПО

От детального проекта — к работающему коду



Детальный проект



Кодирование



Сборка



Отладка



Модульное тестирование



Готовность к интеграции



Что выполняется на стадии реализации



Разработка программных модулей и компонентов



Реализация логики, интерфейсов и обработки ошибок



Создание или уточнение структуры данных и схем БД



Отладка и проверка корректности работы



Модульное тестирование



Что важно соблюдать



Соответствие требованиям и проектным решениям



Понятные интерфейсы модулей и параметров



Документирование ключевых решений



Подготовка к последующей интеграции и приемке



Результат: исходный код, сборка, модульные тесты и сопроводительные материалы, готовые к следующему этапу ЖЦ ПО.

Результаты стадии реализации

Какие артефакты формируются на выходе

ОТ ИДЕИ
К РАБОТАЮЩЕМУ
РЕШЕНИЮ

01



Исходный код

модули, классы, функции,
реализующие требования

02



Сборка / исполняемая версия

приложение, готовое к запуску
или интеграции

03



Модульные тесты

проверка отдельных
компонентов и отчеты
о тестировании

04



Данные и БД

схемы, миграции,
структуры хранения

05



Документация

описание модулей, API,
инструкции для разработчиков
и пользователей

06



Готовность к следующему этапу

основа для интеграции,
системного тестирования
и внедрения



Критерии качества



работоспособность



проверяемость



согласованность
с проектом



понятность
и сопровождаемость

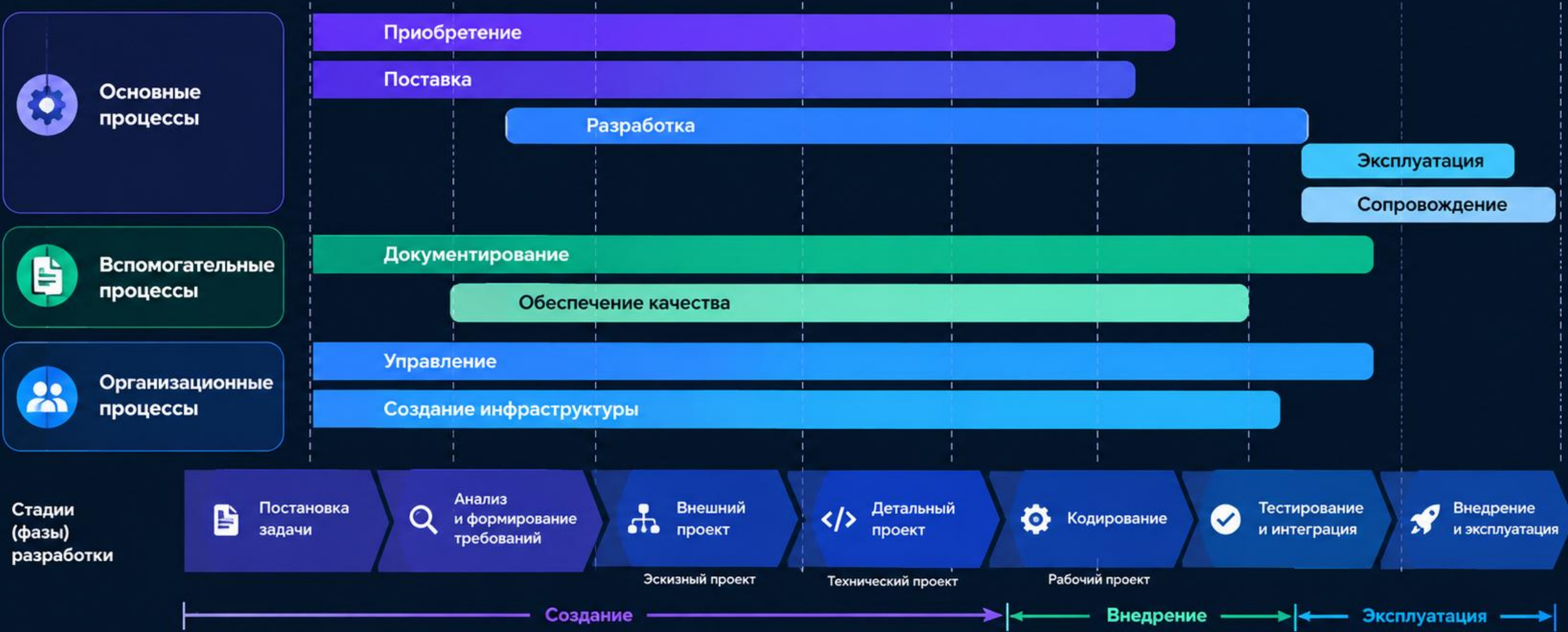


Стадия реализации завершается созданием **работоспособного и проверяемого программного продукта** на уровне компонентов.

КАЧЕСТВО
СЕГОДНЯ —
СТАБИЛЬНОСТЬ
ЗАВТРА

Взаимосвязь между процессами и стадиями разработки в жизненном цикле программного обеспечения

хекслет
колледж



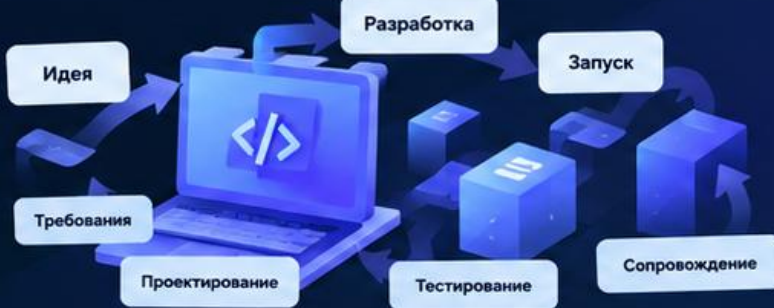
Процессы могут выполняться параллельно и с различной продолжительностью в зависимости от модели ЖЦ, масштаба и особенностей проекта.



Диаграмма показывает, как процессы ЖЦ ПО взаимосвязаны и поддерживают друг друга на различных стадиях.

Жизненный цикл программного обеспечения

От идеи до работающего продукта



"Качественное ПО — это не случайность, а результат правильного процесса."

Жизненный цикл помогает превратить идеи в устойчивые и полезные продукты.

1 Что такое ЖЦ ПО?

Жизненный цикл ПО (ЖЦ ПО) — это совокупность процессов, работ и задач, охватывающих весь период существования программного обеспечения: от формирования идеи до его вывода из эксплуатации.



Модель ЖЦ определяет последовательность, взаимосвязи и структуру работ.

2 Модели ЖЦ ПО

Модель ЖЦ зависит от:



Специфики проекта



Масштаба



Сложности



Условий разработки



Стандарт ISO/IEC 12207 не предписывает конкретную модель ЖЦ, а описывает общий набор процессов.

3 Стадии ЖЦ ПО

- 1 Формирование требований
- 2 Проектирование
- 3 Реализация (разработка)
- 4 Тестирование
- 5 Внедрение
- 6 Эксплуатация
- 7 Сопровождение
- 8 Завершение

4 Взаимосвязь процессов и стадий

Процессы могут выполняться параллельно и с различной продолжительностью в зависимости от модели ЖЦ, масштаба и особенностей проекта.



ЖЦ — это не всегда строгая последовательность. В современных подходах процессы могут повторяться (итерации).

Каждая стадия использует результаты предыдущих стадий и взаимодействует с другими процессами.

5 Детальный обзор стадий ЖЦ ПО



1. Формирование требований

- Сбор и анализ требований
- Определение целей
- Моделирование AS-IS и TO-BE
- Спецификация требований
- Проверка и согласование

Результат:
Спецификация требований



2. Проектирование

- Архитектура системы
- Структура компонентов
- Проектирование данных
- Интерфейсы
- Алгоритмы и логика
- Планирование разработки

Результат:
Проектная документация



3. Реализация

- Кодирование модулей
- Разработка компонент
- Актуализация документации
- Проверка модулей
- Подготовка к тестированию

Результат:
Рабочие модули, код



4. Тестирование

- Проверка функциональности
- Поиск и исправление ошибок
- Проверка соответствия требованиям
- Интеграционное тестирование
- Подготовка к внедрению

Результат:
Протоколы тестирования



5. Внедрение

- Подготовка среды
- Установка и настройка
- Миграция данных
- Обучение пользователей
- Опытная эксплуатация

Результат:
Продолжающее ПО у пользователей



6. Эксплуатация

- Использование ПО
- Мониторинг и контроль
- Обеспечение безопасности
- Поддержка пользователей

Результат:
Стабильная работа системы



7. Сопровождение

- Исправление ошибок
- Адаптация к изменениям
- Оптимизация
- Обновления и новые версии

Результат:
Актуальные и улучшенные версии



8. Завершение

- Вывод из эксплуатации
- Архивация данных
- Документирование
- Оценка результатов

Результат:
Закрытый проект

6 Результаты по всему ЖЦ



Документация



Исходный код



Данные



Тесты



Релизы



Обученные пользователи



Работающая система



Поддержка и обновления

7 Ключевые выводы

- ✓ ЖЦ ПО обеспечивает системный подход к разработке.
- ✓ Качество продукта зависит от правильного выполнения всех стадий.
- ✓ Модель ЖЦ выбирается под конкретный проект.
- ✓ Процессы могут выполняться итеративно и параллельно.
- ✓ Важно документировать результаты каждой стадии.



Правильно организованный жизненный цикл — залог успешного, качественного и востребованного программного продукта!